

INT531

Site Reliability Engineering

Observability II · logs and distributed tracing

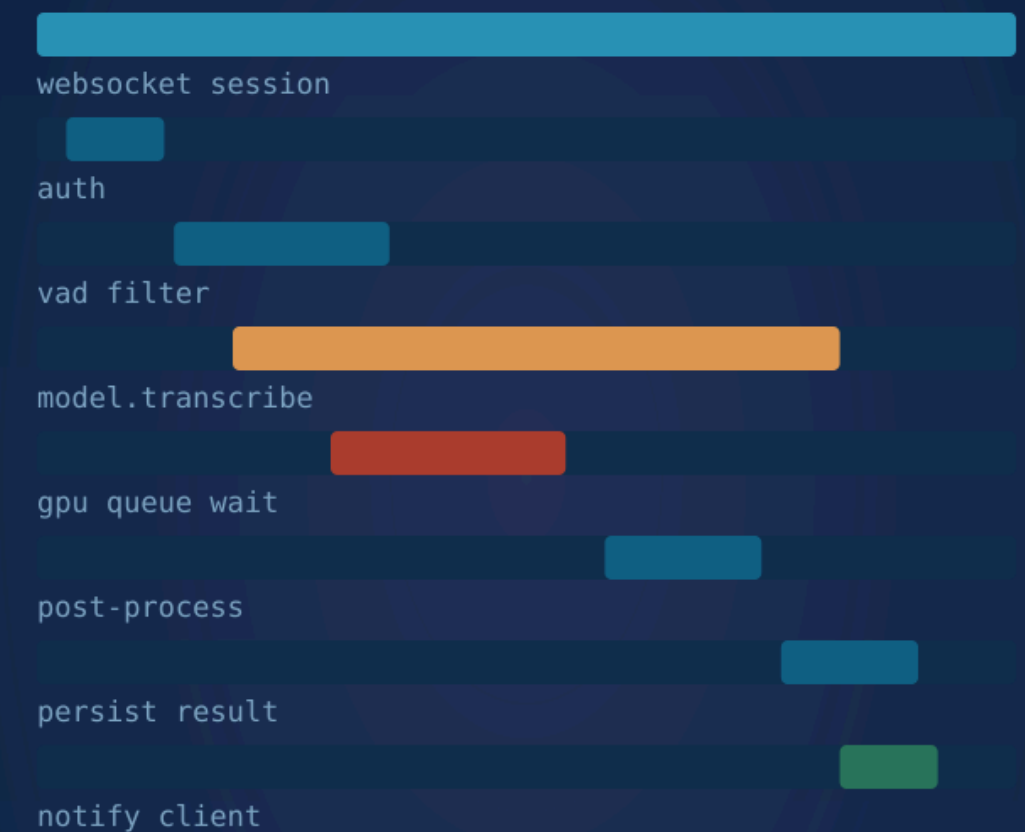
Week 5 · installation day this afternoon

School of Information Technology, King Mongkut's University of Technology Thonburi

Lecture 1.5 h + Field lab 1.5 h | Content updated 2026

trace_id 4bf92f3577b34da6

where did
the time go?



What we cover today

Morning: collecting what metrics cannot say · Afternoon: installing into the rack

01

The gap metrics cannot fill

Why we know 1% failed but not whose request it was

02

Logs that are actually usable

Structured logging, levels, sampling and cost

03

Loki and the label trap

The same rule as Prometheus, missed a second time

04

Distributed tracing

Spans, trace context, OpenTelemetry and sampling

05

Joining the three pillars

One trace_id threading metrics, logs and traces together

06

Lab 5 — installation day

Into the containment rack: mount, cable, power on, accept

Session learning outcomes

By the end of this session you will be able to...

SLO 5.1	Explain what metrics, logs and traces each answer, and choose the right one	CLO3
SLO 5.2	Write structured logs that are searchable, at an appropriate level	CLO3
SLO 5.3	Explain Loki's label trap and design labels that are safe	CLO3, CLO5
SLO 5.4	Read a trace waterfall and say where the time actually went	CLO5
SLO 5.5	Explain trace sampling and its effect on what you can investigate later	CLO3
SLO 5.6	Install equipment to an approved layout and verify it against the acceptance criteria	CLO4, CLO7



01

The gap metrics cannot fill

Metrics say there is a problem; they cannot say to whom, or where

Three questions metrics cannot answer

All three happened to our STT service last week

“Why did this one lecturer's session fail?”

Metrics are aggregates, so we know 1% failed but cannot trace back to that session — because we deliberately kept session_id out of the labels

“Where did those 3.2 seconds go?”

The histogram says 3.2 s in total. It does not say 1.66 s was GPU queue wait and only 0.45 s was actual inference

“What happened just before it broke?”

Metrics sample every 15 s, so the events in between are simply gone — error text, ordering, all of it

None of these is fixed by adding more metrics. They are fixed by collecting a different kind of data

The three pillars of observability

Each is expensive in a different way, and answers a different question

Metrics

How much, how often, how bad

- Cheap: a few bytes per sample
- Aggregated — no individual request
- Cardinality is the hard limit
- Answers questions you prepared for

Logs

What exactly happened, in order

- Expensive: full text per event
- One line per event, keeps detail
- Volume is the hard limit
- Answers 'what did this request do'

Traces

Where the time went, across services

- Sampled: usually 1–10% of requests
- One tree per request, with timings
- Instrumentation effort is the limit
- Answers 'which hop was slow'

The question that picks the pillar

"is the service healthy?" → metrics

"why did this one session fail?" → logs

"which step took the 3 seconds?" → traces

ทั้งสามต้องเชื่อมกันด้วย `trace_id` เดียวกัน ·
ไม่เช่นนั้นจะได้ข้อมูลสามกอง
ที่ประกอบเป็นเรื่องเดียวไม่ได้



02

Logs that are actually usable

A log that reads nicely but cannot be searched is useless at two in the morning

Structured logging — write for the machine

From a human sentence to data with a shape

The unsearchable form

```
logger.info(f"transcribe failed for {sid} after {ms}ms")
```

At 2 a.m. you have to write a regex to pull the numbers out, and the moment somebody edits the wording every filter you built breaks

The searchable form

```
logger.info("transcribe_failed", extra={  
    "session_id": sid, "engine": "whisper-small",  
    "duration_ms": ms, "error": "gpu_timeout",  
    "trace_id": ctx.trace_id})
```

You can filter on fields directly — `error = gpu_timeout`, `duration_ms > 2000` — and the `trace_id` jumps you straight to the trace for that same session

Three things every log line needs

- A UTC timestamp with explicit units, because hosts may sit in different time zones
- A `trace_id`, so you can jump straight to the trace for that same request
- A stable event name such as `transcribe_failed`, not a sentence that changes — filters depend on it

Levels, and how much of each is reasonable

The wrong level makes log storage ten times more expensive for nothing

Level	Use when	Example in the STT service	Expected share
ERROR	Work failed and a human must look	Model failed to load · could not persist a result	under 0.1%
WARN	Abnormal but the system carried on	Queue so deep that audio frames were dropped · a retry happened	under 1%
INFO	Events that must be reconstructable later	Session started · session ended · model loaded	about 5–10%
DEBUG	Detail for chasing a specific problem only	Size of each chunk · parameters passed to the model	off in production

A workable rule: if enabling DEBUG in production multiplies your log volume more than tenfold, the DEBUG lines are in the wrong place — the platform is not at fault

Loki and the label trap

Loki indexes labels only — never the message body



Three LogQL patterns you will use constantly

Same shape as PromQL: select the stream first, then filter the text

Search text within one service

```
{service="live-stt", level="error"}  
  |= "gpu_timeout"
```

Always select the stream by label first, then filter the body with `|=`. Select too broadly and Loki has to read an enormous amount of data

Parse JSON fields and filter numerically

```
{service="live-stt"} | json  
  | duration_ms > 2000
```

This works because the log is structured JSON. Plain prose would need a regex, which is slower and breaks more easily

Turn logs into an error rate

```
sum by (error) (  
  rate({service="live-stt"} | json  
    | level="error" [5m]))
```

LogQL can compute metrics, which is useful before a metric exists. If you need it often, move it to a real metric — it is far cheaper

03

Distributed tracing

Metrics say it is slow, logs say what happened, traces say where the time went

The vocabulary, before anything else

Four terms used throughout

Trace

The whole story of one request, start to finish, held together by a single trace_id

Span

One unit of work inside a trace, with a name, a start, an end and its own span_id

Parent span

The span that called this one, which is what lets the tree be reassembled

Trace context

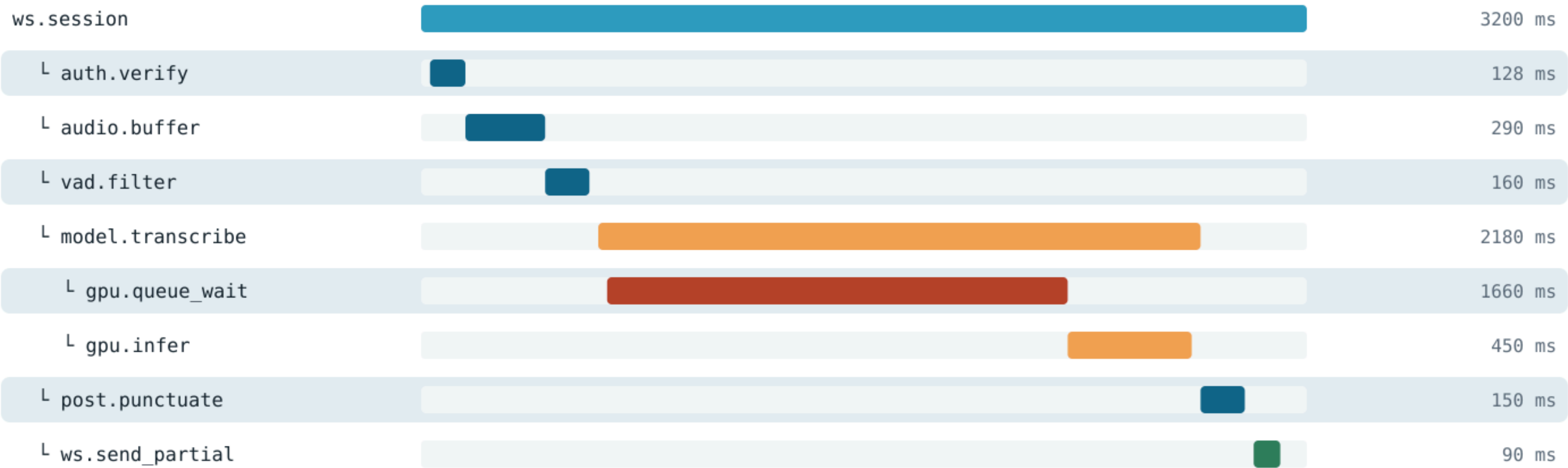
The data passed between services in the traceparent header, per the W3C standard

If trace context is not propagated across a hop, the trace breaks into fragments and you cannot see which layer lost the time

Reading a trace waterfall from the STT service

From the same example we ran in week 4

One streaming session, 3.2 s from first audio frame to first partial transcript



76% ของเวลาไปอยู่ที่ gpu.queue_wait ไม่ใช่ที่การถอดเสียงเอง

เมตริกบอกได้แค่ p99 คือ 3.2 วินาที · Trace บอกว่า 1.66 วินาทีในนั้นคือการรอคิว GPU
ซึ่งแก้ด้วยการเพิ่ม worker ไม่ใช่เปลี่ยนโมเดล

Trace sampling

You cannot keep every request, but sampling badly leaves you without the ones you needed

Approach	How it works	Advantage	The cost you accept
Head sampling	Decide at the first span, e.g. keep 1%	Simple, with predictable cost	Failed requests usually are not kept, since the decision came before the outcome
Tail sampling	Wait for the request to finish, then decide on the outcome	Keeps the slow and failed requests reliably	Requires buffering in memory and more configuration
Rate limiting	Cap traces per second per service	Protects cost when traffic spikes	The worst incidents are exactly when most traces get dropped

A practical starting point: head sampling at 1–5% plus a rule that always keeps failed requests, then consider tail sampling once the system justifies it

Joining the three pillars

Without the join you have three piles of data that cannot be assembled into one story

1

Put trace_id on every log line

When a log line looks wrong, jump straight to the trace for that request

2

Attach exemplars to histogram metrics

Click an outlier on the graph and open the trace of the request that caused it

3

Use the same service name everywhere

The service label must be spelled identically in all three systems, or nothing joins up

4

Keep clocks in sync with NTP

Two seconds of clock skew puts events in the wrong order, and you will diagnose the wrong cause

The last one sounds trivial and has misled countless teams — skewed clocks make a later event look like the earlier one



04

Lab 5 — installation day

The BOM checked out · CR-01 is approved · today the equipment goes into the rack

Conditions before installation starts

Any one missing and that group cannot begin

Documents that must be approved

- A signed CR-01 stating the window and the rollback plan
- An AR-01 for install day naming everyone
- A signed PWR-01 with the power check completed
- The approved RACK-01 target layout
- CAB-01, the cabling schedule with label codes

Materials ready on site

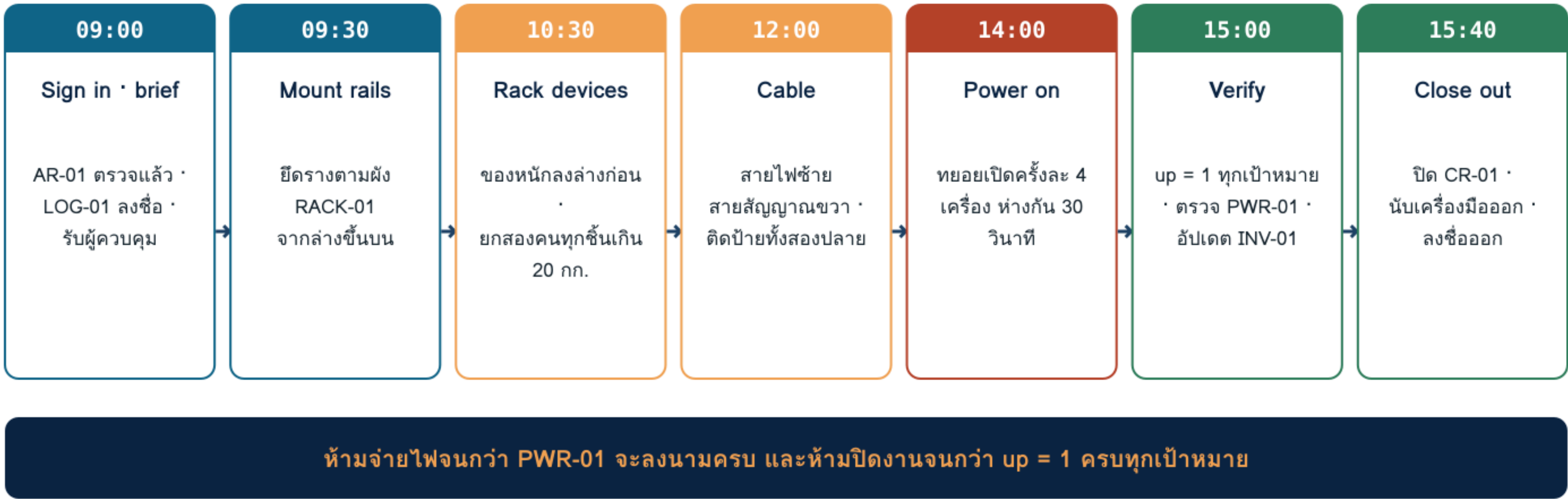
- Equipment sorted and tagged already
- Rail kits in the quantity from the BOM
- Power and data cables cut to length
- Cable labels pre-written with their codes
- Blanking panels for every remaining gap

What you must know before touching anything

- The bottom-up installation order, and why
- Which items are a two-person lift
- The power-on order, and why it is staggered
- All six acceptance criteria
- The conditions that stop the work and trigger rollback

The installation-day sequence

The steps you cannot skip, and where you stop for a signature



Why installation runs bottom to top

An ordering that looks trivial and is very expensive to correct later

Heavy items go in low first

The UPS and storage sit at the bottom, lowering the centre of gravity and the risk of tipping while rails are extended

Work upward from there

Fit the top first and every heavy item afterwards has to be lifted past installed equipment

Leave the reserved U empty as drawn

Those gaps exist for the servers still to arrive. Fill them now and the whole rack gets stripped again later

Fit blanking panels as soon as a U is known to be empty

Left for later, nobody comes back to do it, and the containment does not perform as designed

Cable after everything is mounted, not as you go

Cabling in parallel tangles the runs, and removing one device later drags the neighbours' cables with it

Acceptance criteria before close-out

All six must pass for the installation to count as finished

☐	Every device sits at the U position on the approved RACK-01	Technical reviewer
☐	Every empty U is capped with a blanking panel	Safety officer
☐	Every cable is labelled at both ends and matches CAB-01	Inventory lead
☐	Measured load per PDU is within 80% of its rating	Electrical staff
☐	Prometheus reports up = 1 for every target	Change owner
☐	INV-01 updated with new locations and CR-01 closed	Change owner

The fifth criterion is what joins the morning to the afternoon — the installation is not finished until the monitoring system can see what was installed

Controls that apply on installation day

This is executing an approved plan, not deciding on the spot

A.8.32	Change management	Work inside the CR-01 window; any change of plan needs fresh approval
A.7.2	Physical entry control	Entry under AR-01, with an escort throughout
A.7.8	Equipment siting and protection	Install to RACK-01; positions are not changed on site
A.7.11	Supporting utilities	Measure actual load after power-on and record it on PWR-01
A.7.12	Cabling security	Power and signal on separate trays, both ends labelled
A.5.9	Inventory of assets	Update every item's new location in INV-01 the same day

Assignments and next week

Week 6: alerting and on-call

Group work, due in 5 days

- Photographs of the finished rack, front and rear
- INV-01 updated with every new location
- PWR-01 with the load measured after power-on
- A closed CR-01 with a summary of anything that did not go to plan

Individual, before the next session

- Read Observability Engineering, chapters 6–7 on structured events
- Complete pre-class quiz 5 on the LMS
- Add at least three structured log lines to the sample app, including `trace_id`
- Submit a screenshot of one trace with at least three levels of spans

Prepare for next week

- Add Alertmanager to the week 4 stack
- Draft the alerts you think your own service should have
- Bring a view on which alerts are worth waking someone for

The question that opens next session: an alert that fires at 2 a.m. and needs no action — should it be deleted, or kept just in case?

References

Split by part of the session

For the lecture

- Majors, C., Fong-Jones, L., & Miranda, G. (2022). Observability Engineering. O'Reilly. Ch. 4–8
- OpenTelemetry documentation — traces, context propagation and sampling
- W3C Trace Context Recommendation — the traceparent header format
- Grafana Labs — Loki best practices on label design
- Sigelman, B. et al. (2010). Dapper, a Large-Scale Distributed Systems Tracing Infrastructure. Google

For the field work

- ISO/IEC 27001:2022 — Annex A: A.5.9, A.7.2, A.7.8, A.7.11, A.7.12, A.8.32
- The manufacturer's installation guides for the rack and blade chassis in use
- The course forms pack — CR-01, AR-01, PWR-01, RACK-01, CAB-01, INV-01
- Your group's BOM, with the capacity checks passed, from week 4

Today in three sentences

01

Metrics say there is a problem, logs say what happened, traces say where the time went — none of them substitutes for another

02

Loki's labels follow exactly the same rule as Prometheus: `session_id` and `trace_id` belong in the message body, never in a label

03

This afternoon's installation is not finished until the monitoring system can see the machines that were installed

Week 6 · Alerting and on-call — once the system can see everything, when do you wake someone?

A note on the artwork (instructor slide)

Not for projection

Artwork in this deck

The title image, the three-pillar chart, the trace waterfall, the log pipeline and the install sequence were drawn as vectors for this course and can be edited freely

The numbers in the trace waterfall

They follow the structure of the STT service actually run in week 4, but the per-span timings are teaching examples, not measurements

A caution under A.7.4

Do not publish install-day photographs showing asset tags, network diagrams or screens with credentials outside the course